

DOSSIER DE PROJET

Titre Professionnel - Développeur Web et Web Mobile

EasyFit

Application e-commerce de vêtements de sport

Candidat: BERTIL Guillaume

Centre de formation: Centre Européen de Formation

Session: 2026

Titre visé: Développeur Web et Web Mobile - Niveau 5

1 - Liste des compétences mise en oeuvre.....	4
Livrables.....	5
2 - Expression des besoins.....	5
2.1 Contexte et origine du projet.....	5
2.2 Objectifs du projet.....	6
2.3 Limites du projet.....	6
2.4 Public cible.....	7
2.5 Contraintes du projet.....	7
2.6 Environnement humain.....	9
3 - Environnement technique.....	10
3.1 Stack technique.....	10
3.2 Architecture de l'application.....	10
3.3 Stratégie Git et Github.....	11
4 - Réalisations.....	13
4.1 CP1 Installer et configurer son environnement de travail.....	13
Installation de l'environnement.....	13
Configuration des variables d'environnement.....	13
Gestion des versions avec Git.....	14
4.2 CP2 Maquettage des interfaces utilisateur.....	14
Outil de maquettage.....	14
Charte graphique.....	14
Maquettes réalisées.....	15
Schéma d'enchaînement des maquettes.....	16
Accessibilité et RGAA.....	17
4.3 CP3 Réaliser des interfaces utilisateur statiques web ou web mobile.....	17
Moteur de templates Twig.....	17
Intégration de Tailwind CSS.....	18
Responsive et approche mobile-first.....	18
Sémantique HTML et accessibilité RGAA.....	21
Pages statiques.....	21
4.4 CP4 Développer la partie dynamique des interfaces utilisateur web ou web mobile.....	21
Approche technique.....	21
Menu burger — JavaScript vanilla.....	22
Sélection de variante produit — formulaire HTML natif.....	23
Badge panier dynamique — extension Twig.....	24
4.5 CP5 Mettre en place une base de données relationnelle.....	26
Système de gestion de base de données.....	26
Modélisation des données.....	26
Catalogue produit.....	26
Panier.....	26
Commandes.....	27
Avis.....	27
Contraintes d'intégrité.....	27

MCD (Modèle Conceptuel de Données).....	28
MLD (Modèle Logique de Données).....	28
4.6 CP6 Développer des composants d'accès aux données SQL et NoSQL.....	30
ORM Doctrine et pattern Repository.....	30
Requêtes personnalisées avec le QueryBuilder.....	31
ProductRepository.....	31
Utilisation dans les contrôleurs.....	32
Persistance des données.....	33
4.7 CP7 Développer des composants métier côté serveur.....	33
Entités Doctrine et mapping objet-relationnel.....	33
Logique métier dans les entités.....	34
Validation des données.....	35
Authentification Symfony.....	36
Formulaires Symfony.....	37
Enum OrderStatus.....	39
4.8 CP8 Documenter le déploiement d'une application dynamique web ou web mobile.....	39
Prérequis.....	39
Variables d'environnement.....	40
Procédure de déploiement.....	40
Configuration du serveur web.....	41
Contrôle d'accès.....	41
Documentation complémentaire.....	41
4.9 Jeu d'essai.....	42
4.10 Veille sécurité.....	43
Sources consultées.....	43
Vulnérabilités identifiées et correctifs appliqués.....	44
5 - Conclusion.....	45
6 - Annexe.....	46
Page catalogue :.....	46
Fiche produit :.....	47
Panier avec articles :.....	48
Page de confirmation de commande :.....	48
Interface EasyAdmin :.....	48
Script de création de la base de données SQL :.....	49

1 - Liste des compétences mise en oeuvre

Le projet EasyFit, réalisé dans le cadre de la formation, permet de mettre en oeuvre l'ensemble des huit compétences professionnelles du titre professionnel Développeur Web et Web Mobile, réparties sur les deux activités types.

Activité type	CP	Compétence professionnelle	☐
AT1 - Front-end	CP1	Installer et configurer son environnement de travail	☒
	CP2	Maquetter des interfaces utilisateur web ou web mobile	☒
	CP3	Réaliser des interfaces utilisateur statiques web ou web mobile	☒
	CP4	Développer la partie dynamique des interfaces utilisateur web ou web mobile	☒
AT2 - Back-end	CP5	Mettre en place une base de données relationnelle	☒

	CP6	Développer des composants d'accès aux données SQL et NoSQL	✓
	CP7	Développer des composants métier côté serveur	✓
	CP8	Documenter le déploiement d'une application dynamique web ou web mobile	✓

Livrables

- **Dépôt GitHub** - code source versionné avec stratégie de branches : <https://github.com/guillaumebertil/easyfit>
- **Maquettes Figma** - interface desktop et mobile : <https://www.figma.com/design/za1E5DjYUHPDiS7mL3M4w0/Untitled?node-id=0-1&p=f&t=zBe2innNXMpuyEM9-0>
- **Dossier projet** - Document de présentation du projet et compétences mise en oeuvre
- **Diaporama de présentation** - support de soutenance
- **Procédure de déploiement** - documentation d'installation et de mise en production (CP8 et README)

2 - Expression des besoins

2.1 Contexte et origine du projet

EasyFit est un projet personnel de site e-commerce spécialisé dans la vente de vêtements de sport. Il a été conçu et développé intégralement dans le cadre de la formation au titre professionnel Développeur Web et Web Mobile, au Centre Européen de Formation.

Le choix de ce projet répond à une volonté de couvrir l'intégralité des compétences du référentiel dans un contexte réaliste et représentatif du métier : gestion d'un catalogue produit, système d'authentification, espace client, tunnel de commande, administration back-office, et respect des bonnes pratiques de sécurité et d'accessibilité.

2.2 Objectifs du projet

L'application EasyFit doit permettre à l'utilisateur de :

- Parcourir un catalogue de vêtements de sport organisé par catégories
- Consulter le détail d'un produit (déclinaisons taille/couleur, images, avis clients)
- Gérer son panier (ajout, modification de quantité, suppression d'articles)
- Créer un compte, se connecter et consulter son historique de commande
- Passer une commande via un tunnel de commande sécurisé
- Déposer un avis avec une note de 1 à 5 sur les produits achetés
- L'application doit permettre à un administrateur de :
 - Gérer l'ensemble des entités du catalogue via une interface d'administration (EasyAdmin)
- Gérer les utilisateurs et leurs rôles
- Consulter et gérer les commandes

2.3 Limites du projet

Les fonctionnalités listées ci-dessous sont identifiées comme hors périmètre du projet actuel et constituent des évolutions prévues.

Les fonctionnalités suivantes ne sont pas implémentées dans la version actuelle et sont documentées comme évolutions futures:

- Paiement en ligne (intégration Stripe)
- Gestion du stock liée au paiement
- Wishlist (liste de souhaits)
- Sous-catégories de produits
- Envoi d'emails transactionnels (Symfony Mailer)

Le tunnel de commande est fonctionnel mais simule la validation du paiement sans appel à un prestataire de paiement réel.

2.4 Public cible

L'application s'adresse à deux types d'utilisateurs:

- Les clients finaux: tout utilisateur souhaitant acheter des vêtements de sport en ligne, sur ordinateur ou mobile
- Les administrateurs: les gestionnaires de la boutique disposant d'un accès au back-office EasyAdmin

2.5 Contraintes du projet

Contrainte	Détail
Technique	Utilisation du framework Symfony 7 avec le moteur de template Twig
Style	Interface responsive conçue avec Tailwind CSS via symfonycasts/tailwind-bundle
Base de données	MySQL / MariaDB, gestion locale via XAMPP
Versioning	Dépôt GitHub avec stratégie de branches par fonctionnalité (feature/xxx)
Sécurité	Application de bonnes pratiques inspirées des recommandations OWASP et ANSSI , protection CSRF, validation des entrées, sécurisation des routes
Accessibilité	Prise en compte des recommandations RGAA (balise sémantiques, contrastes, navigation clavier)
RGPD	Mentions légales, page CGV, recueil du consentement à l'inscription

2.6 Environnement humain

EasyFit est un projet individuel réalisé en autonomie complète dans le cadre de la formation au titre professionnel Développeur Web et Web Mobile au Centre Européen de Formation.

En tant que seul développeur du projet, j'ai assuré l'ensemble des rôles:

- **Analyse des besoins** - définition des fonctionnalités, du public cible et des contraintes
- **Conception** - modélisation de la base de données, maquettage des interfaces avec Figma
- **Développement front-end** - intégration des interfaces avec Twig et Tailwind CSS
- **Développement back-end** - développement des entités, repositories, contrôleurs et formulaires avec Symfony
- **Tests** - réalisation du jeu d'essai et correction des bugs identifiés
- **Documentation** - rédaction du dossier de projet et du README

L'organisation du travail a suivi une approche itérative par fonctionnalité, chaque feature étant développée sur une branche Git dédiée puis fusionnée sur la branche principale via une pull request.

3 - Environnement technique

3.1 Stack technique

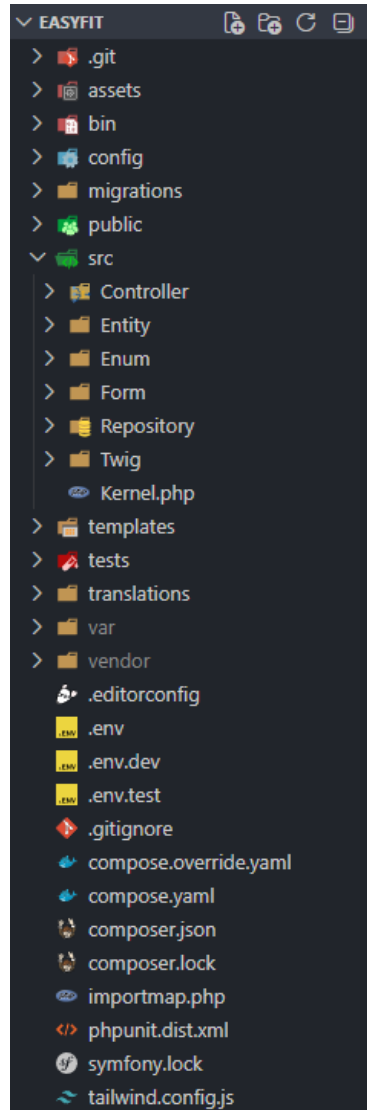
Catégorie	Technologie	Version / Détail
Front-end	Twig	Moteur de template Symfony
	Tailwind CSS	Via symfonycasts/tailwind-bundle
Back-end	PHP / Symfony	PHP 8.5.4 / Symfony 7
	Doctrine / ORM	Mapping objet-relationnel, migrations
	EasyAdmin Bundle	Interface d'administration CRUD
Base de données	MySQL / MariaDB	Serveur local XAMPP
Maquettage	Figma	Maquettes desktop et mobile
Versionning	Git / GitHub	Stratégie de branches
IDE	VS Code	VS Code 1.119.1
Qualité de code	PHPStan	Vérification des normes PSR-12

3.2 Architecture de l'application

EasyFit suit l'architecture MVC (Modèle-Vue-Contrôleur) imposée par le framework Symfony. L'organisation du projet respecte la structure conventionnelle du framework:

- src/Controller: contrôleurs gérant les requêtes HTTP et la logique du routage
- src/Entity: entités Doctrine représentant le modèle de données
- src/Repository: accès aux données via l'ORM Doctrine
- src/Form: formulaires Symfony typés
- src/Security: gestion de l'authentification et des autorisations

- templates/: vues Twig organisées par fonctionnalité



3.3 Stratégie Git et Github

Le dépôt GitHub est organisé selon une stratégie de branches par fonctionnalité. Chaque nouvelle fonctionnalité est développée sur une branche dédiée (feature/auth, feature/catalogue, feature/panier, etc.) puis fusionnée sur la branche main via une pull request après vérification.

☐ 0 Open ☒ 18 Closed		Author -	Label -	Projects -	Milestones -	Reviews -	Assignee -	Sort -
☐	🔗 refactor: commentaires du code #36 by guillaumbertil Owner was merged yesterday					👁 1		
☐	🔗 feat: seo (balises meta, robots.txt et sitemap.xml) #34 by guillaumbertil Owner was merged yesterday					👁 1		
☐	🔗 feat: finitions du frontend #32 by guillaumbertil Owner was merged yesterday					👁 1		
☐	🔗 fix: gestion de l'erreur quand la variante produit n'existe pas #30 by guillaumbertil Owner was merged last week					👁 1		
☐	🔗 feat: sécurisation des routes (cart, order) #28 by guillaumbertil Owner was merged last week					👁 1		
☐	🔗 feat: système d'avis (Review, formulaire, affichage) #26 by guillaumbertil Owner was merged last week					👁 1		
☐	🔗 feat: pages statiques (mentions légales, CGV, contact) #24 by guillaumbertil Owner was merged 2 weeks ago					👁 1		
☐	🔗 feat: historique des commandes #22 by guillaumbertil Owner was merged 2 weeks ago					👁 1		
☐	🔗 feat: tunnel de commande (Order, OrderItem, confirmation) #20 by guillaumbertil Owner was merged 2 weeks ago					👁 1		
☐	🔗 feat: panier complet (ajout, modification, suppression) #18 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feat: page catalogue et fiche produit #16 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feat: entités Product, Size, Color, ProductImage, ProductVariant + CR... #14 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feat: entité Category, CRUD EasyAdmin et navigation dynamique #12 by guillaumbertil Owner was merged 3 weeks ago							
☐	🔗 feat: espace utilisateur (affichage et modification du profil) #10 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feat: installation de EasyAdmin + CRUD User #8 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feat: système d'authentification complet (inscription, connexion, déc... #6 by guillaumbertil Owner was merged 3 weeks ago					👁 1		
☐	🔗 feature: homepage, header, navbar et footer #4 by guillaumbertil Owner was merged last month					👁 1		
☐	🔗 chore: installation et configuration de Tailwind CSS #2 by guillaumbertil Owner was merged last month					👁 1		

Lien vers le dépôt GitHub: <https://github.com/guillaumbertil/easyfit>

4 - Réalisations

Cette partie présente les réalisations du projet EasyFit, organisées par compétence professionnelle.

4.1 CP1 Installer et configurer son environnement de travail

Installation de l'environnement

L'environnement de développement a été configuré localement pour répondre aux besoins du projet Symfony 7. Les outils suivants ont été installés et configurés:

- PHP 8.5.4
- Symfony CLI 5.16.1 pour le serveur de développement local
- Composer 2.9.5 pour la gestion des dépendances PHP
- Git 2.52.0 pour le versioning connecté à GitHub
- XAMPP pour le serveur MySQL/MariaDB local
- VS Code 1.119.1 comme environnement de développement intégré

```
$ php -v && symfony -v && composer -v && git --version
PHP 8.5.4 (cli) (built: Mar 10 2026 23:30:42) (ZTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Built by The PHP Group
Zend Engine v4.5.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
Symfony CLI version 5.16.1 (c) 2021-2026 Fabien Potencier (2025-11-25T07:30:20Z - stable)
Composer version 2.9.5 2026-01-29 11:40:53
PHP version 8.5.4 (C:\Users\guill\scoop\apps\php\current\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
git version 2.52.0.windows.1
```

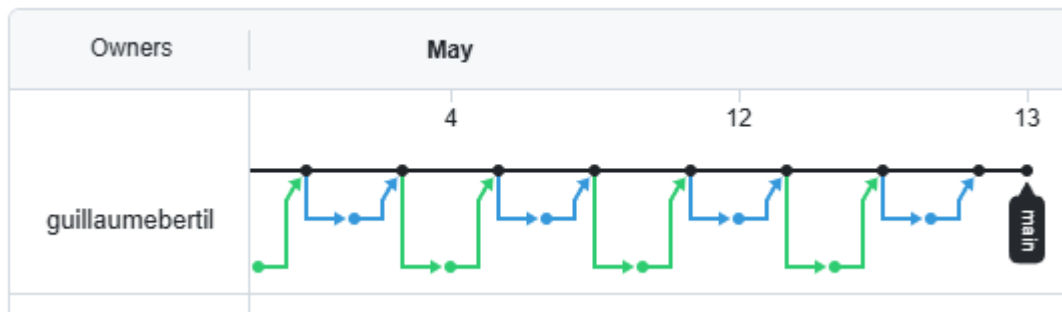
Configuration des variables d'environnement

Les variables d'environnement sont réparties entre plusieurs fichiers selon la convention Symfony. Le fichier `.env` versionné contient les valeurs par défaut non sensibles. Les secrets (APP_SECRET) sont placés dans `.env.local`, qui n'est pas versionné grâce au `.gitignore`.

```
###> symfony/framework-bundle ###
APP_ENV=dev
APP_SECRET=
DATABASE_URL="mysql://root@127.0.0.1:3306/easyfit?serverVersion=10.4.32-MariaDB&charset=utf8mb4"
```

Gestion des versions avec Git

Le dépôt GitHub est organisé selon une stratégie de branches par fonctionnalité. Chaque fonctionnalité est développée sur une branche dédiée (feature/auth, feature/catalogue, feature/panier, etc.) puis fusionnée sur la branche main via une pull request après vérification. Le projet compte 18 pull requests fusionnés.



4.2 CP2 Maquettage des interfaces utilisateur

Outil de maquettage

Les maquettes ont été réalisées avec Figma. Deux versions ont été conçues pour chaque interface principale: une version desktop (1440px) et une version mobile (375px), conformément à l'approche mobile-first.

Charte graphique

La charte graphique d'EasyFit s'articule autour d'une identité visuelle sportive et moderne:

Couleurs



Primaire: #16A31A



Secondaire: #111827



Fond: #F9F9FB



Texte courant: #374151



Accentuation: #15803D

Typographie

H1 - Inter Bold 32px

H2 - Inter Bold 24px

H3 - Inter Semi Bold 18px

Corps - Inter Regular 16 px

Petit texte - Inter Regular 14 px

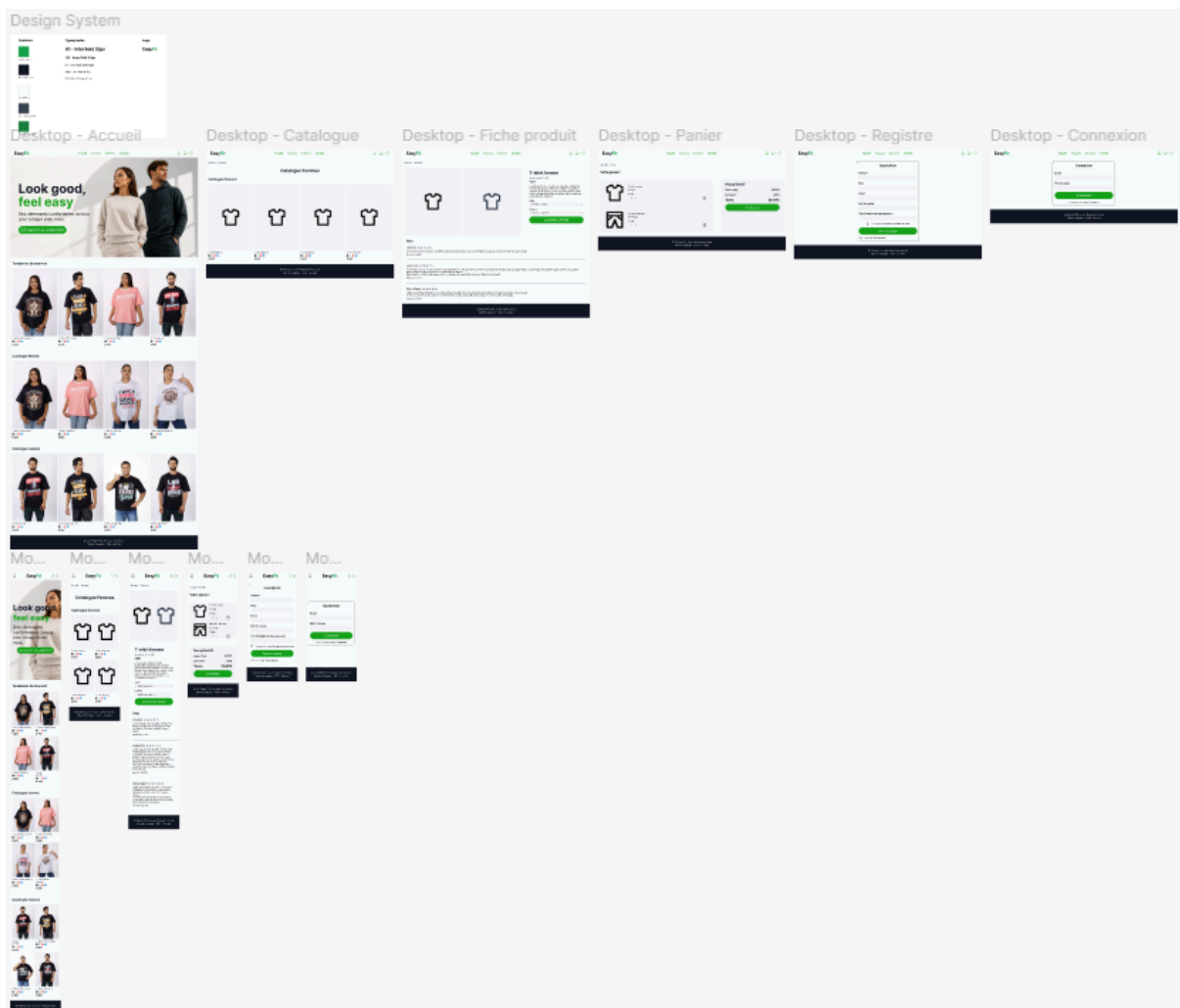
Logo

EasyFit

Maquettes réalisées

Les interfaces suivantes ont fait l'objet d'un maquettage complet en version desktop et mobile:

- Page d'accueil
- Catalogue
- Fiche produit
- Panier
- Connexion / Inscription

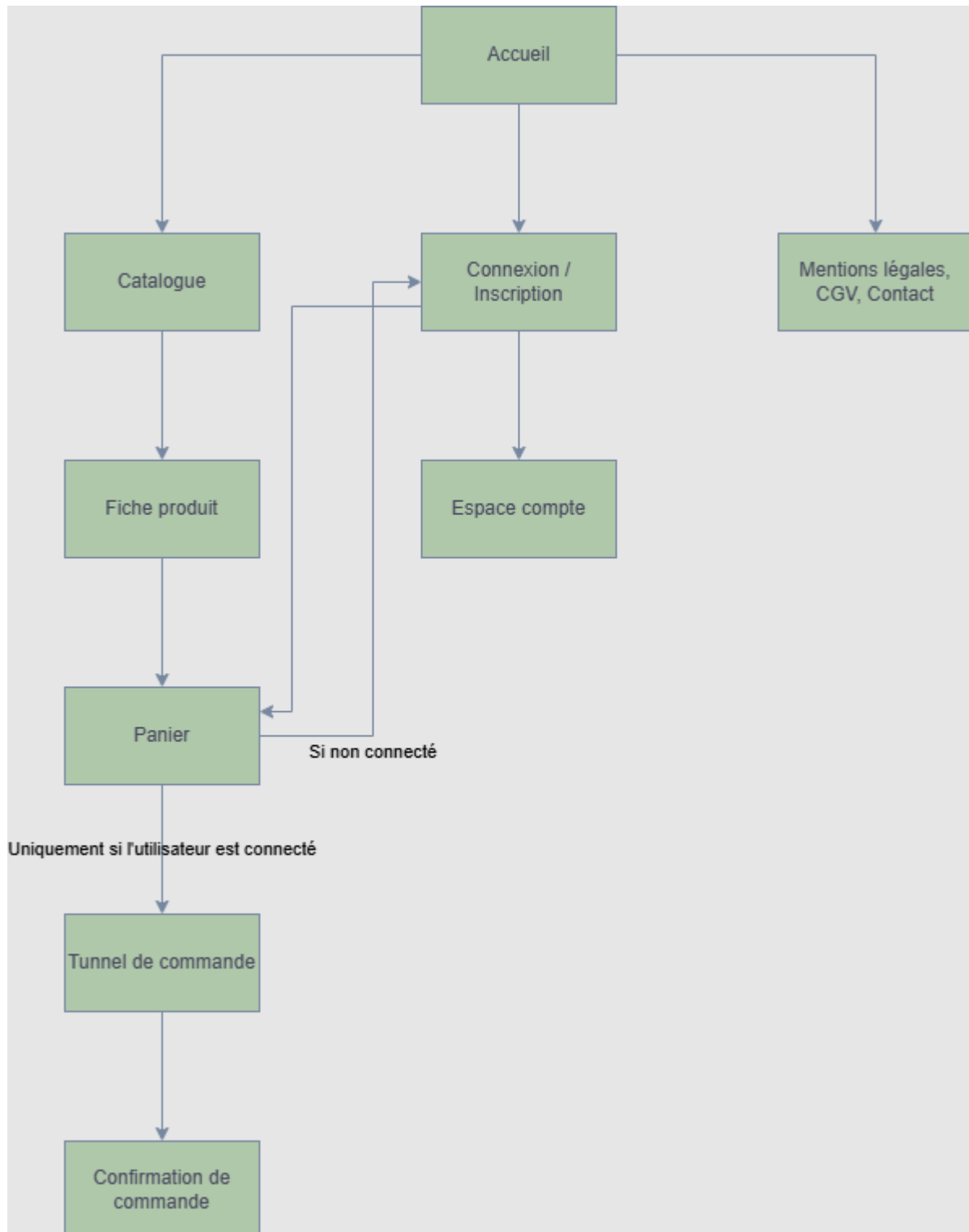


Lien vers le projet Figma:

<https://www.figma.com/design/za1E5DjYUHPDiS7mL3M4w0/Untitled?node-id=0-1&p=f&t=L2ZLFdE4aGPu4jbC-0>

Schéma d'enchaînement des maquettes

Le schéma ci-dessous représente les différents écrans de l'application et leurs liens de navigation. Il illustre le parcours utilisateur depuis l'accueil jusqu'à la confirmation de commande, ainsi que les conditions d'accès aux pages sécurisées.



Accessibilité et RGAA

Les maquettes ont été conçues en tenant compte des recommandations du RGAA:

- Contrastes de couleurs suffisants entre le texte et le fond
- Taille minimale des zones cliquables adaptée aux équipements tactiles
- Hiérarchie visuelle claire avec des titres structurés (H1, H2, H3)
- Navigation cohérente sur toutes les pages

4.3 CP3 Réaliser des interfaces utilisateur statiques web ou web mobile

Moteur de templates Twig

Les interfaces d'EasyFit sont construites avec le moteur de templates Twig, intégré nativement à Symfony. L'organisation des vues repose sur un système d'héritage: un template de base `base.html.twig` définit la structure commune à toutes les pages (balises `<head>`, navbar, footer, chargement des assets Tailwind), et chaque page étend ce gabarit via la directive `{% extends %}` en surchargeant les blocs définis (`{% block title %}`, `{% block body %}`).

Les composants récurrents (navbar, footer, carte produit) sont isolés dans des templates partiels inclus via `{% include %}`, ce qui garantit la cohérence visuelle et facilite la maintenance.

```
<body class="min-h-screen flex flex-col">
  {% include "partials/_header.html.twig" %}

  <main class="flex flex-1 flex-col items-center w-full text-secondary bg-surface">
    <div class="w-full max-w-screen-2xl mt-4">
      {% block body %}{% endblock %}
    </div>
  </main>

  {% include "partials/_footer.html.twig" %}
</body>
```

Intégration de Tailwind CSS

Le style de l'application est entièrement géré avec Tailwind CSS, intégré via le bundle `symfonycasts/tailwind-bundle`. Ce bundle compile les classes Tailwind à la volée en environnement de développement via la commande `symfony console tailwind:build --watch`, et produit un fichier CSS optimisé pour la production.

Le projet utilise `Tailwind CSS v4`, dont la configuration passe par le fichier CSS principal via la directive `@theme`. La charte graphique d'EasyFit y est déclarée:

```
assets > styles > css app.css > button
1  @import "tailwindcss";
2
3  @theme {
4      --color-primary: #16A31A;
5      --color-secondary: #111827;
6      --color-accent: #15803D;
7      --color-surface: #F9FAFB;
8      --color-text-main: #374151;
9      --font-sans: "Inter", sans-serif;
10 }
```

Ces variables génèrent automatiquement les classes utilitaires correspondantes (`text-primary`, `bg-accent`, `text-secondary`, etc.), directement utilisables dans les templates Twig.

Responsive et approche mobile-first

L'interface est entièrement responsive, conçue selon une approche mobile-first conforme aux maquettes Figma. Les breakpoints Tailwind (`lg:`) permettent d'adapter la mise en page selon la taille de l'écran.

La mise en page des listes de produits repose sur une grille CSS via les classes Tailwind `grid` et `grid-cols-*` : deux colonnes sur mobile, quatre colonnes sur desktop.

```
{# Catalogue femme #}
<section class="px-4 my-8">
  <h2 class="text-2xl font-bold mb-5"><a href="{{ path('app_product', {categorySlug: womenCategory.slug }) }}">Catalogue femme</a></h2>
  <div class="grid grid-cols-2 lg:grid-cols-4">
    {% for product in womenProducts %}
      {% include 'partials/_product_card.html.twig' with {'product': product} %}
    {% endfor %}
  </div>
</section>
```

Version desktop

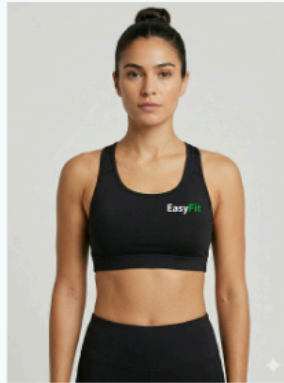


Tendances du moment



T-shirt femme

● ○
9,99 €



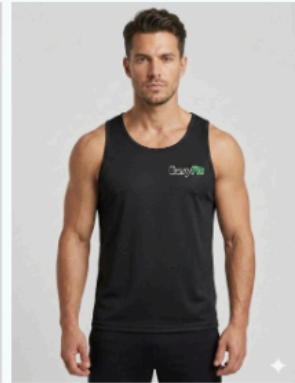
Brassière de sport

● ○
14,99 €



T-shirt homme

● ○
9,99 €



Débardeur homme

● ○
14,99 €

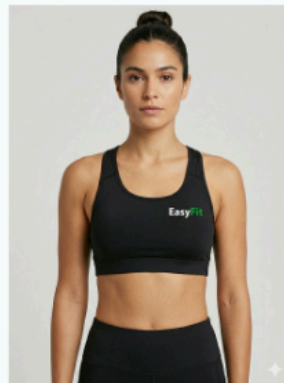
Catalogue femme



T-shirt femme



T-shirt femme manches

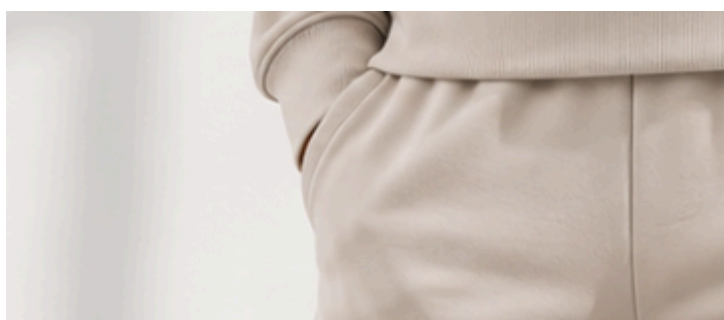


Brassière de sport



Sweat femme

Version mobile



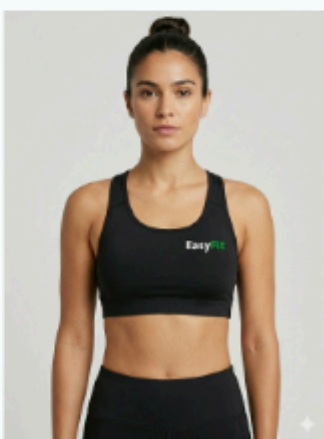
Tendances du moment



T-shirt femme



9,99 €



Brassière de sport



14,99 €



T-shirt homme



9,99 €



Débardeur homme



14,99 €

Catalogue femme

Sémantique HTML et accessibilité RGAA

Les templates respectent les recommandations du RGAA en matière de sémantique HTML:

- Utilisation des balises structurales appropriées : `<header>`, `<nav>`, `<main>`, `<footer>`, `<section>`, `<article>`
- Hiérarchie de titres respectée sur chaque page (`<h1>` unique, `<h2>`, `<h3>` cohérents)
- Attributs `alt` renseignés sur toutes les images produits
- Navigation clavier fonctionnelle grâce aux éléments interactifs natifs (`<a>`, `<button>`, `<input>`)
- Contrastes de couleurs conformes entre le texte et le fond
- Attributs `aria-label` ajoutés sur les icônes sans texte visible (panier, compte utilisateur)

Pages statiques

Les pages statiques (mentions légales, CGV, contact) sont intégrées en HTML sémantique via des templates Twig dédiés, accessibles depuis le footer de toutes les pages. Ces pages participent au respect des obligations légales identifiées dans les contraintes du projet (RGPD, mentions obligatoires).

4.4 CP4 Développer la partie dynamique des interfaces utilisateur web ou web mobile

Approche technique

La dynamique des interfaces d'EasyFit repose principalement sur le cycle requête/réponse HTTP géré par Symfony : les actions utilisateur (ajout au panier, modification de quantité, passage de commande, dépôt d'avis) déclenchent des soumissions de formulaires traitées côté serveur, suivies d'une redirection. Cette approche garantit la cohérence des données sans nécessiter de JavaScript complexe.

Le seul script JavaScript utilisé côté client gère l'ouverture et la fermeture du menu burger sur mobile.

Menu burger — JavaScript vanilla

Le menu de navigation mobile est contrôlé par un script JavaScript vanilla placé dans `_navbar.html.twig`, encapsulé dans une **IIFE** (Immediately Invoked Function Expression) pour éviter les collisions de variables dans le scope global.

Le script gère quatre comportements:

- Une vérification de l'existence des éléments DOM au chargement, via un **return** anticipé si `burger-btn` ou `mobile-menu` sont absents
- Un clic sur le bouton burger bascule les classes **hidden** et **flex** sur le menu mobile, et met à jour **aria-expanded**
- Un clic en dehors du menu le ferme via un listener sur **document**
- Un clic sur un lien du menu le ferme avant la navigation

Le script gère également la compatibilité avec Turbo (le système de navigation sans rechargement de Symfony) : le listener sur **document** est stocké dans `window._navbarDocClickListener` et retiré avant d'être réenregistré, ce qui évite les doublons lors des navigations successives.

```
(function () {
  const menuBtn = document.getElementById('burger-btn');
  const mobileMenu = document.getElementById('mobile-menu');

  if (!menuBtn || !mobileMenu) return;

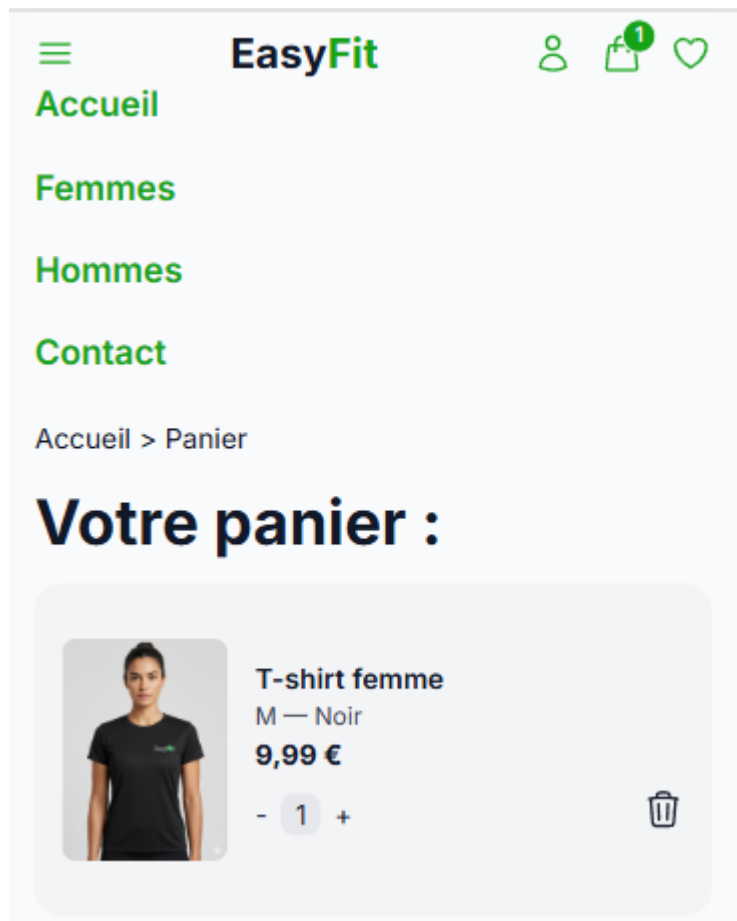
  const closeMenu = () => {
    mobileMenu.classList.add('hidden');
    mobileMenu.classList.remove('flex');
    menuBtn.setAttribute('aria-expanded', 'false');
  };

  menuBtn.addEventListener('click', (e) => {
    e.stopPropagation();
    const isHidden = mobileMenu.classList.contains('hidden');
    if (isHidden) {
      mobileMenu.classList.remove('hidden');
      mobileMenu.classList.add('flex');
    } else {
      closeMenu();
    }
    menuBtn.setAttribute('aria-expanded', String(!mobileMenu.classList.contains('hidden')));
  });

  mobileMenu.addEventListener('click', (e) => {
    e.stopPropagation();
  });

  mobileMenu.querySelectorAll('a').forEach(link => {
    link.addEventListener('click', closeMenu);
  });

  // Nettoie le listener précédent pour éviter les doublons lors des navigations Turbo
  if (window._navbarDocClickListener) {
    document.removeEventListener('click', window._navbarDocClickListener);
  }
  window._navbarDocClickListener = closeMenu;
  document.addEventListener('click', closeMenu);
})();
```



Sélection de variante produit - formulaire HTML natif

Sur la fiche produit, l'utilisateur sélectionne la taille et la couleur via deux éléments `<select>` HTML natifs. Les options sont générées dynamiquement par Twig en itérant sur les collections `size` et `color` associées au produit. À la soumission, les identifiants sélectionnés sont envoyés en `POST` au `CartController`, qui recherche la `ProductVariant` correspondante en base de données.

```

{# Formulaire d'ajout au panier #}
<form action="{{ path('app_cart_add') }}" method="POST">

    {# Sélection de la taille #}
    <div class="flex flex-col mb-4">
        <label for="size-selector" class="font-semibold mb-1">Taille :</label>
        <select name="size" id="size-selector" class="w-full bg-gray-100 rounded-lg px-3 py-1">
            <option value="">-- Choisir une taille --</option>
            {% for size in product.size %}
                <option value="{{ size.id }}">{{ size }}</option>
            {% endfor %}
        </select>
    </div>

    {# Sélection de la couleur #}
    <div class="flex flex-col mb-4">
        <label for="color-selector" class="font-semibold mb-1">Couleur :</label>
        <select name="color" id="color-selector" class="w-full bg-gray-100 rounded-lg px-3 py-1">
            <option value="">-- Choisir une couleur --</option>
            {% for color in product.color %}
                <option value="{{ color.id }}">{{ color }}</option>
            {% endfor %}
        </select>
    </div>

    <input type="hidden" name="product" value="{{ product.id }}">
    <button type="submit" class="w-full text-white font-semibold bg-primary mt-4 px-4 py-2 rounded-full hover:bg-accent">
        AJOUTER AU PANIER
    </button>
</form>

```

Badge panier dynamique - extension Twig

Le badge affichant le nombre d'articles dans le panier est mis à jour automatiquement à chaque rechargement de page. Il est alimenté par la fonction `getCartCount()` définie dans `AppExtension`, une extension Twig personnalisée. Cette fonction récupère le panier de l'utilisateur connecté via le `CartRepository`, puis additionne les quantités de chaque `CartItem` pour retourner le total exact.

```

/** Retourne le nombre d'articles dans le panier de l'utilisateur connecté, 0 s'il n'est pas connecté */
#[AsTwigFunction('getCartCount')]
public function getCartCount(): int
{
    // Récupérer l'utilisateur
    $user = $this->security->getUser();
    if (!$user) {
        return 0;
    }

    // Récupérer le panier de l'utilisateur
    $cart = $this->cartRepository->findOneBy(['user' => $user]);
    if (!$cart) {
        return 0;
    }

    // Récupérer les items du panier
    $cartItems = $cart->getCartItems();

    $totalQuantity = 0;

    foreach ($cartItems as $cartItem) {
        $totalQuantity += $cartItem->getQuantity();
    }

    // Retourner la quantité totale des articles du panier
    return $totalQuantity;
}

```

Le badge est affiché dans la navbar via `{% set cartCount = getCartCount() %}` et n'apparaît que si le total est supérieur à zéro.

```

{% set cartCount = getCartCount() %}
{% if cartCount > 0 %}
    <span class="absolute -top-2 -right-2 flex justify-center
        {{ cartCount }}">
    </span>
{% endif %}

```

4.5 CP5 Mettre en place une base de données relationnelle

Système de gestion de base de données

La base de données d'EasyFit repose sur MySQL/MariaDB, administrée localement via XAMPP. La gestion du schéma est entièrement prise en charge par Doctrine ORM, l'outil de mapping objet-relationnel intégré à Symfony. Les tables sont générées et versionnées via le système de migrations Doctrine, ce qui garantit la traçabilité des évolutions du schéma tout au long du développement.

- Créer la base de données
`symfony console doctrine:database:create`
- Générer une migration après modification des entités:
`symfony console make:migration`
- Appliquer les migrations en base:
`symfony console doctrine:migrations:migrate`

Modélisation des données

La base de données compte **13 tables métier**, organisées autour de quatre domaines fonctionnels : le catalogue produit, la gestion des utilisateurs et de l'authentification, le panier et les commandes, et les avis clients.

Catalogue produit

La table `product` est reliée à `category` par une clé étrangère (`category_id`). Chaque produit peut être décliné en plusieurs variantes via la table `product_variant`, qui croise un produit, une taille et une couleur. Les relations `ManyToMany` entre `product` et `size` d'une part, et `product` et `color` d'autre part, sont matérialisées par les tables de jointure `product_size` et `product_color`. Les images sont stockées dans `product_image`, liée à `product` par une relation `OneToMany`.

Panier

Chaque utilisateur possède au plus un panier (`cart`), lié par une relation `OneToOne` avec `user`. Le panier contient des articles (`cart_item`), chacun référant une `product_variant` et une quantité.

Commandes

La table `order` enregistre chaque commande avec son total, sa date de création et son statut (géré via un Enum `OrderStatus`). Les lignes de commande sont stockées dans `order_item`, qui conserve le prix unitaire au moment de l'achat afin d'être insensible aux variations de prix ultérieures.

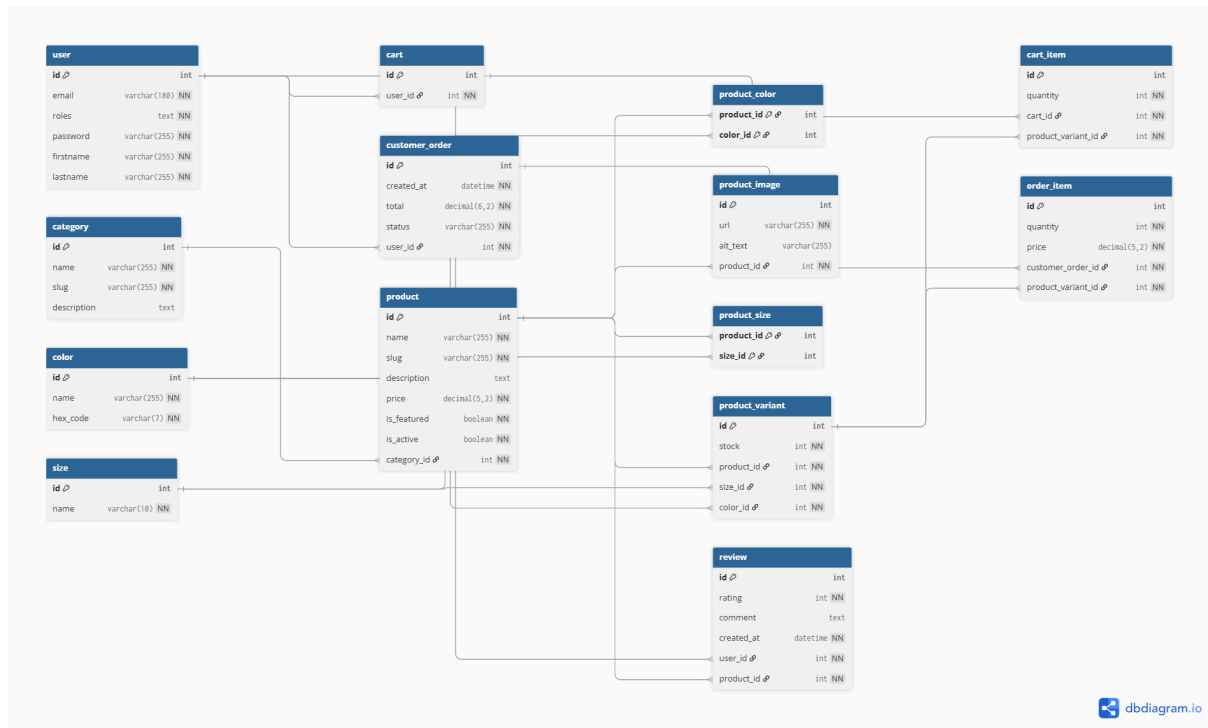
Avis

La table `review` relie un utilisateur à un produit, avec une note (1 à 5), un commentaire optionnel et une date de création.

Contraintes d'intégrité

Les contraintes de clés étrangères sont définies au niveau de la base de données et gérées automatiquement par Doctrine. Les suppressions en cascade sont configurées sur les tables de jointure (`product_size`, `product_color`) pour assurer la cohérence des données lors de la suppression d'un produit, d'une taille ou d'une couleur. Les champs à unicité contrainte (`email` dans `user`, `slug` dans `product` et `category`) sont déclarés avec des index `UNIQUE`.

MCD (Modèle Conceptuel de Données)



MLD (Modèle Logique de Données)

USER (id, email, roles, password, firstname, lastname)

- Clé primaire : id
- Colonne unique : email

CATEGORY (id, name, slug, description)

- Clé primaire : id
- Colonne unique : slug

PRODUCT (id, name, slug, description, price, is_featured, is_active, #category_id)

- Clé primaire : id
- Clé étrangère : category_id → CATEGORY(id)

- Colonne unique : slug

PRODUCT_IMAGE (id, url, alt_text, #product_id)

- Clé primaire : id
- Clé étrangère : product_id → PRODUCT(id)

SIZE (id, name)

- Clé primaire : id

COLOR (id, name, hex_code)

- Clé primaire : id

PRODUCT_SIZE (#product_id, #size_id)

- Clé primaire : (product_id, size_id)
- Clés étrangères : product_id → PRODUCT(id), size_id → SIZE(id)

PRODUCT_COLOR (#product_id, #color_id)

- Clé primaire : (product_id, color_id)
- Clés étrangères : product_id → PRODUCT(id), color_id → COLOR(id)

PRODUCT_VARIANT (id, stock, #product_id, #size_id, #color_id)

- Clé primaire : id
- Clés étrangères : product_id → PRODUCT(id), size_id → SIZE(id), color_id → COLOR(id)

CART (id, #user_id)

- Clé primaire : id
- Clé étrangère : user_id → USER(id) CASCADE
- Colonne unique : user_id

CART_ITEM (id, quantity, #cart_id, #product_variant_id)

- Clé primaire : id
- Clés étrangères : cart_id → CART(id) CASCADE, product_variant_id → PRODUCT_VARIANT(id)

ORDER (id, created_at, total, status, #user_id)

- Clé primaire : id
- Clé étrangère : user_id → USER(id) CASCADE

ORDER_ITEM (id, quantity, price, #customer_order_id, #product_variant_id)

- Clé primaire : id
- Clés étrangères : customer_order_id → ORDER(id) CASCADE, product_variant_id → PRODUCT_VARIANT(id)

REVIEW (id, rating, comment, created_at, #user_id, #product_id)

- Clé primaire : id
- Clés étrangères : user_id → USER(id) CASCADE, product_id → PRODUCT(id)

4.6 CP6 Développer des composants d'accès aux données SQL et NoSQL

ORM Doctrine et pattern Repository

L'accès aux données dans EasyFit est entièrement géré via **Doctrine ORM**, l'outil de mapping objet-relationnel intégré à Symfony. Doctrine permet de manipuler les données sous forme d'objets PHP sans écrire de SQL brut, en s'appuyant sur le pattern **Repository** pour centraliser et organiser les requêtes.

Chaque entité dispose de son propre repository, qui étend **ServiceEntityRepository**. Les méthodes natives héritées (**find**, **findBy**, **findOneBy**, **findAll**) couvrent les cas simples. Pour les besoins spécifiques de l'application, des méthodes personnalisées ont été développées via le **QueryBuilder** de Doctrine.

Requêtes personnalisées avec le QueryBuilder

ProductRepository

```
/**
 * Retourne les produits actifs d'une catégorie
 *
 * @return Product[]
 */
public function findActiveByCategory(Category $category): array
{
    return $this->createQueryBuilder('p')
        ->andWhere('p.category = :category')
        ->andWhere('p.isActive = true')
        ->setParameter('category', $category)
        ->getQuery()
        ->getResult();
}

/**
 * Retourne les produits mis en avant
 *
 * @return Product[]
 */
public function findFeatured(): array
{
    return $this->createQueryBuilder('p')
        ->andWhere('p.isFeatured = true')
        ->getQuery()
        ->getResult();
}
```

OrderRepository

```

/**
 * Retourne les commandes d'un utilisateur
 *
 * @return Order[]
 */
public function findByUser(User $user): array
{
    return $this->createQueryBuilder('o')
        ->andWhere('o.user = :user')
        ->setParameter('user', $user)
        ->getQuery()
        ->getResult();
}

```

CartRepository

```

/**
 * Retourne le panier d'un utilisateur
 *
 * @return ?Cart
 */
public function findByUser(User $user): ?Cart
{
    return $this->createQueryBuilder('c')
        ->andWhere('c.user = :user')
        ->setParameter('user', $user)
        ->getQuery()
        ->getOneOrNullResult();
}

```

Utilisation dans les contrôleurs

Les contrôleurs n'appellent que les méthodes des repositories — ils ne contiennent aucune logique de requête. Par exemple dans `HomeController`:

```
// Récupérer les produits mis en avant (isFeatured = true)
$featuredProducts = $productRepository->findFeatured();

// Récupérer les produits actifs pour chaque genre
$womenProducts    = $productRepository->findActiveByCategory($womenCategory);
$menProducts      = $productRepository->findActiveByCategory($menCategory);
```

Persistence des données

Les opérations d'écriture (création, modification, suppression) sont gérées via l'`EntityManager` de Doctrine. Le cycle persist/flush est appliqué systématiquement : `persist()` déclare l'entité à Doctrine, `flush()` exécute les requêtes SQL correspondantes en base de données.

```
if (!$cart) {
    $cart = new Cart();
    $cart->setUser($user);

    $entityManager->persist($cart);
    $entityManager->flush();
}
```

4.7 CP7 Développer des composants métier côté serveur

Entités Doctrine et mapping objet-relationnel

Les entités Symfony constituent le cœur du modèle de données d'EasyFit. Chaque entité est une classe PHP annotée avec les attributs Doctrine qui définissent le mapping vers la base de données.

L'entité `Product` illustre les différents types de relations gérés par Doctrine : une relation `ManyToOne` vers `Category`, des relations `ManyToMany` vers `Size` et `Color` (matérialisées par des tables de jointure), et des relations `OneToMany` vers `ProductImage`, `ProductVariant` et `Review`.

```
#[ORM\ManyToOne(inversedBy: 'products')]
#[ORM\JoinColumn(nullable: false)]
private ?Category $category = null;

/**
 * @var Collection<int, Size>
 */
#[ORM\ManyToMany(targetEntity: Size::class, inversedBy: 'products')]
private Collection $size;

/**
 * @var Collection<int, Color>
 */
#[ORM\ManyToMany(targetEntity: Color::class, inversedBy: 'products')]
private Collection $color;

/**
 * @var Collection<int, ProductImage>
 */
#[ORM\OneToMany(targetEntity: ProductImage::class, mappedBy: 'product')]
private Collection $productImages;
```

Logique métier dans les entités

L'entité **Product** encapsule deux méthodes métier permettant de calculer la note moyenne et le nombre d'avis d'un produit directement depuis l'objet, sans requête supplémentaire:

```
/**
 * Calcule la note moyenne des avis
 *
 * @return float La moyenne (0 si aucun avis)
 */
public function getAverageRating(): float
{
    if ($this->reviews->isEmpty()) {
        return 0;
    }

    $total = 0;

    foreach ($this->reviews as $review) {
        $total += $review->getRating();
    }

    return $total / count($this->reviews);
}

/**
 * Renvoie le nombre d'avis sur un produit
 *
 * @return int Nombre d'avis
 */
public function getReviewsCount(): int
{
    return $this->reviews->count();
}
```

Ces méthodes sont appelées directement dans les templates Twig via `product.averageRating` et `product.reviewsCount`.

Validation des données

Les contraintes de validation sont déclarées directement dans les entités via les attributs du composant `Validator` de Symfony. L'entité `User` en illustre l'utilisation:

```
#[ORM\Column(length: 180)]
#[Assert\NotBlank()]
#[Assert\Email(
    message: 'L\'adresse e-mail n\'est pas valide'
)]
private ?string $email = null;
```

Email

L'adresse e-mail n'est pas valide

Mot de passe

La validation est déclenchée automatiquement lors de la soumission des formulaires Symfony via `$form->isValid()`.

Authentification Symfony

L'entité `User` implémente les interfaces `UserInterface` et `PasswordAuthenticatedUserInterface` de Symfony Security. Elle définit l'identifiant de connexion (`getUserIdentifier` retourne l'email), la gestion des rôles avec `ROLE_USER` attribué par défaut, et le stockage du mot de passe hashé.

```

/**
 * A visual identifier that represents this user.
 *
 * @see UserInterface
 */
public function getUserIdentifier(): string
{
    return (string) $this->email;
}

/**
 * @see UserInterface
 */
public function getRoles(): array
{
    $roles = $this->roles;
    // guarantee every user at least has ROLE_USER
    $roles[] = 'ROLE_USER';

    return array_unique($roles);
}

```

Formulaires Symfony

Les formulaires sont construits avec le composant Form de Symfony, qui lie automatiquement les données soumises aux entités correspondantes.

Le formulaire `ReviewType` permet à un utilisateur de déposer un avis avec une note de 1 à 5 et un commentaire optionnel:

```

$builder
    ->add('rating', ChoiceType::class, [
        'label' => 'note',
        'choices' => [
            '1 étoile' => 1,
            '2 étoiles' => 2,
            '3 étoiles' => 3,
            '4 étoiles' => 4,
            '5 étoiles' => 5
        ]
    ])
    ->add('comment', TextareaType::class, [
        'required' => false,
        'label' => 'Votre avis',
        'attr' => [
            'placeholder' => 'Laissez votre avis sur ce produit'
        ]
    ])

```

Le traitement dans le contrôleur suit le cycle standard Symfony :

```
// Traiter la requête (lire les données POST soumises)
$form->handleRequest($request);

if ($form->isSubmitted() && $form->isValid()) {
    // Associer l'avis à l'utilisateur connecté et au produit
    $review->setUser($user);
    $review->setProduct($product);
    $review->setCreatedAt(new \DateTimeImmutable());

    $entityManager->persist($review);
    $entityManager->flush();

    // Rediriger vers la même fiche produit pour éviter la re-soumission du formulaire
    return $this->redirectToRoute('app_product_show', [
        'categorySlug' => $product->getCategory()->getSlug(),
        'slug' => $product->getSlug(),
    ]);
}
```

Le champ `plainPassword` utilise l'option `hash_property_path` de Symfony, qui hash automatiquement le mot de passe saisi et l'écrit dans la propriété `password` de l'entité `User` — sans aucun traitement manuel dans le contrôleur.

```
->add('plainPassword', RepeatedType::class, [
    'type' => PasswordType::class,
    'first_options' => [
        'label' => 'Mot de passe',
        'attr' => [
            'placeholder' => '*****',
        ],
        'constraints' => [
            new Assert\NotBlank(),
            new Assert\Length([
                'min' => 6,
                'minMessage' => 'Le mot de passe doit contenir au minimum {{ limit }} caractères',
                'max' => 16,
                'maxMessage' => 'Le mot de passe ne doit pas dépasser {{ limit }} caractères'
            ]),
            new Assert\Regex([
                'pattern' => '/[A-Z]/',
                'message' => 'Le mot de passe doit contenir au moins une majuscule'
            ])
        ],
        'hash_property_path' => 'password',
    ],
    'second_options' => [
        'label' => 'Confirmation du mot de passe',
        'attr' => [
            'placeholder' => '*****',
        ],
    ],
    'mapped' => false,
```

Enum OrderStatus

Le statut d'une commande est géré via un **Enum** PHP natif à valeur chaîne. Il centralise les quatre états possibles du cycle de vie d'une commande et expose une méthode **getLabel()** retournant le libellé en français pour l'affichage côté client :

```
enum OrderStatus: string
{
    case PENDING = 'pending';
    case PAID = 'paid';
    case SHIPPED = 'shipped';
    case CANCELLED = 'cancelled';

    public function getLabel(): string
    {
        return match ($this) {
            OrderStatus::PENDING => 'En attente de paiement',
            OrderStatus::PAID => 'Payée',
            OrderStatus::SHIPPED => 'Expédiée',
            OrderStatus::CANCELLED => 'Annulée',
        };
    }
}
```

4.8 CP8 Documenter le déploiement d'une application dynamique web ou web mobile

Prérequis

Le déploiement d'EasyFit nécessite un environnement serveur disposant des éléments suivants :

- PHP 8.2 minimum avec les extensions pdo_mysql, intl, mbstring, xml
- Composer 2.x
- MySQL ou MariaDB
- Symfony CLI (optionnel mais recommandé)
- Un serveur web Apache ou Nginx avec support du URL rewriting

Variables d'environnement

Les variables d'environnement sensibles ne sont pas versionnées. Elles doivent être définies dans un fichier `.env.local` à la racine du projet, en s'appuyant sur le fichier `.env` comme modèle :

```
cp .env .env.local
```

Les variables à renseigner obligatoirement :

```
APP_ENV=prod
APP_SECRET=une_chaine_aleatoire_de_32_caracteres
DATABASE_URL="mysql://user:password@host:3306/easyfit?serverVersion=
10.4.32-MariaDB&charset=utf8mb4"
```

`APP_SECRET` doit être une chaîne aléatoire unique, générée via:

```
php -r "echo bin2hex(random_bytes(16));"
```

Procédure de déploiement

1. Cloner

```
git clone https://github.com/guillaumbertil/easyfit.git
cd easyfit
```

2. Installer les dépendances PHP

```
composer install --no-dev --optimize-autoloader
```

3. Créer la base de données et exécuter les migrations

```
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate --no-interaction
```

4. Compiler les assets Tailwind

```
php bin/console tailwind:build
```

5. Vider et préchauffer le cache

```
php bin/console cache:clear
```

```
php bin/console cache:warmup
```

Configuration du serveur web

Le point d'entrée de l'application est `public/index.php`. Le serveur web doit rediriger toutes les requêtes vers ce fichier via le fichier `.htaccess` présent dans le dossier `public/` :

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteRule ^(.*)$ index.php [QSA,L]
</IfModule>
```

Contrôle d'accès

Zone	Rôle requis
/admin	ROLE_ADMIN
/account , /cart , /order	ROLE_USER
Catalogue, pages statiques	Public

Documentation complémentaire

La documentation complète d'installation est disponible dans le fichier `README.md` du dépôt GitHub : <https://github.com/guillaumebertil/easyfit>

4.9 Jeu d'essai

Le jeu d'essai suivant recense les scénarios testés sur l'application EasyFit, avec les données d'entrée, les actions effectuées et les résultats obtenus.

	Scénario	Données d'entrée	Action	Résultat attendu	Résultat obtenu
1	Inscription valide	Prénom, nom, email valide, mot de passe Test123, CGV cochés	Soumettre le formulaire	Compté créé, utilisateur connecté, redirection accueil	Conforme
2	Inscription email dupliqué	Email déjà en base	Soumettre le formulaire	Message "Cette adresse e-mail est déjà utilisée"	Conforme
3	Inscription mot de passe sans majuscule	Mot de passe: test123	Soumettre le formulaire	Message "doit contenir au moins une majuscule"	Conforme
4	Connexion valide	Email et mot de passe correct	Soumettre le formulaire	Redirection, accueil, utilisateur connecté	Conforme
5	Connexion mauvais mot de passe	Email correct, mauvais mot de passe	Soumettre le formulaire	Message "identifiants invalides"	Conforme
6	Affichage catalogue		Clic sur une catégorie	Liste des produits actifs	Conforme
7	Affichage fiche		Clic sur un	Détail avec	Conforme

	produit		produit	tailles, couleurs, et avis	
8	Ajout au panier	Taille et couleur sélectionnées	Clic "Ajouter au panier"	Article ajouté, badge mis à jour	Conforme
9	Modification quantité		Clic "-" ou "+"	Quantité et total mis à jour	Conforme
10	Suppression article		Clic "Supprimer"	Article retiré du panier	Conforme
11	Accès panier sans connexion		Accès à /cart	Redirection vers login	Conforme
12	Passage commande	Panier non vide, connecté	Clic "Commander"	Commande créée, panier vidé, confirmation	Conforme
13	Dépôt d'un avis	Note 4, commentaire	Soumettre le formulaire	Avis affiché sur la fiche produit	Conforme
14	Accès avis sans connexion		Accès à la fiche produit	Formulaire non affiché, lien de connexion	Conforme
15	Accès admin ROLE_ADMIN	Compte admin	Accès /admin	Interface EasyAdmin	Conforme
16	Accès admin ROLE_USER	Compte utilisateur	Accès /admin	Accès refusé	Conforme

4.10 Veille sécurité

Dans le cadre du développement d'EasyFit, une veille sécurité a été effectuée en s'appuyant sur les ressources suivantes :

Sources consultées

- **OWASP Top 10** (owasp.org) - liste des 10 vulnérabilités web les plus critiques
- **ANSSI** (ssi.gouv.fr) - recommandations de l'Agence Nationale de la Sécurité des Systèmes d'Information

- **Documentation Symfony Security** - bonnes pratiques intégrées au framework

Vulnérabilités identifiées et correctifs appliqués

Vulnérabilité	Risque	Correctif appliqué
Injection SQL	Accès non autorisé aux données	Utilisation de Doctrine ORM, pas de SQL Brut
XSS (Cross-Site-Scripting)	Injection de scripts malveillants	Échappement automatique des variables TWIG
CSRF (Cross-Site Request Forgery)	Actions non autorisées au nom de l'utilisateur	Protection CSRF activée par défaut sur tous les formulaires Symfony
Mot de passe en clair	Vol de credentials	Hachage via <code>hash_property_path</code> et l'algorithme bcrypt
Accès non autorisé aux routes	Élévation de privilèges	Sécurisation des routes dans <code>security.yaml</code> avec <code>ROLE_USER</code> et <code>ROLE_ADMIN</code>
Exposition des secrets	Fuite de données sensibles	Variables sensibles dans <code>.env.local</code> non versionné

5 - Conclusion

Le projet EasyFit m'a permis de mettre en œuvre l'ensemble des compétences du titre professionnel Développeur Web et Web Mobile dans un contexte réaliste et complet. Du maquettage Figma jusqu'au développement back-end avec Symfony 7, en passant par la modélisation de la base de données et la sécurisation des routes, chaque étape du cycle de développement a été couverte.

Ce projet m'a également permis d'adopter de bonnes pratiques professionnelles : versioning avec Git et stratégie de branches, séparation des responsabilités entre contrôleurs et repositories, documentation du code via PHPDoc, et respect des recommandations RGAA et OWASP.

Plusieurs évolutions sont envisagées pour la suite : intégration d'un prestataire de paiement Stripe, gestion du stock liée aux commandes, envoi d'emails transactionnels, et déploiement en production.

6 - Annexe

Page catalogue :

EasyFit

Accueil Femmes Hommes Contact



Accueil > Femmes

Femmes

Découvrez la collection Femme EasyFit : des vêtements alliant confort et style pour toutes vos activités. Que ce soit pour le sport ou le quotidien, profitez de coupes modernes et de matières respirantes pensées pour vous accompagner au mieux.

Catalogue Femmes



T-shirt femme

☒ ☐

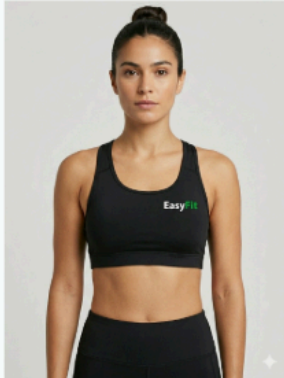
9,99 €



T-shirt femme manches longues

☒ ☐

11,99 €



Brassière de sport

☒ ☐

14,99 €



Sweat femme

☒ ☐

19,99 €

© 2026 EasyFit | Tous les droits sont réservés
Mentions légales - CGV - Contact

46

Fiche produit :

EasyFit

[Accueil](#) [Femmes](#) [Hommes](#) [Contact](#)

Accueil > Femmes



T-shirt femme

☆☆☆☆ 0 avis

9,99 €

T-shirt de sport femme au design sobre et moderne, conçu dans une matière légère et respirante pour un confort optimal pendant l'entraînement. Sa coupe ajustée assure une grande liberté de mouvement, idéale pour le fitness, la musculation ou le quotidien.

Taille :

-- Choisir une taille --

Couleur :

-- Choisir une couleur --

AJOUTER AU PANIER

Panier avec articles :

EasyFit[Accueil](#)[Femmes](#)[Hommes](#)[Contact](#)

Accueil > Panier

Votre panier :



T-shirt femme
M — Noir
9,99 €

- 2 +





T-shirt homme
M — Noir
9,99 €

- 1 +



Récapitulatif

Sous-total : 29,97 €

Livraison : **Gratuit**

Total : 29,97 €

Commander

Page de confirmation de commande :

EasyFit[Accueil](#)[Femmes](#)[Hommes](#)[Contact](#)

Accueil

Confirmation de votre commande

Interface EasyAdmin :

Easyfit

Rechercher

Super Administrateur

Dashboard

Utilisateurs

Categories

Produits

Déclinaisons

Images

Tailles

Couleurs

Avis

Produits

Créer Produit

☐	Name	Slug	Category	Price	Size	Color	Is Featured	Is Active	
☐	T-shirt femme	t-shirt-femme	Femmes	9,99	4	2	☒	☒	...
☐	T-shirt femme manches longues	t-shirt-femme-manches-longues	Femmes	11,99	4	2	☐	☒	...
☐	Brassière de sport	brassiere-de-sport	Femmes	14,99	4	2	☒	☒	...
☐	Sweat femme	sweat-femme	Femmes	19,99	4	2	☐	☒	...
☐	T-shirt homme	t-shirt-homme	Hommes	9,99	4	2	☒	☒	...
☐	T-shirt homme manches longues	t-shirt-homme-manches-longues	Hommes	11,99	4	2	☐	☒	...
☐	Débardeur homme	debardeur-homme	Hommes	14,99	4	2	☒	☒	...
☐	Sweat homme	sweat-homme	Hommes	19,99	4	2	☐	☒	...

8 résultats

Script de création de la base de données SQL :

```
CREATE TABLE `cart` (  
  `id` int(11) NOT NULL,  
  `user_id` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
-----  
  
--  
-- Structure de la table `cart_item`  
--  
  
CREATE TABLE `cart_item` (  
  `id` int(11) NOT NULL,  
  `quantity` int(11) NOT NULL,  
  `cart_id` int(11) NOT NULL,  
  `product_variant_id` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
-----  
  
--  
-- Structure de la table `category`  
--  
  
CREATE TABLE `category` (  
  `id` int(11) NOT NULL,  
  `name` varchar(255) NOT NULL,  
  `slug` varchar(255) NOT NULL,  
  `description` longtext DEFAULT NULL  
)
```

Le script SQL complet est disponible dans le dépôt GitHub :

<https://github.com/guillaumebertil/easyfit>